

CHAPTER 2

LANGUAGE MODELLING

CHAPTER OVERVIEW

The domain of language is quite vast. It presents an almost infinite number of sentences to the reader (or computer). To handle such a large number of sentences, we have to create a model of the domain, which can subsequently be simplified and handled computationally. A number of language models have been proposed. We introduce some of these models in this chapter. To create a general model of any language is a difficult task. There are two approaches for language modelling. One is to define a grammar that can handle the language. The other is to capture the patterns in a grammar language statistically. This chapter has a mixed approach. It gives a glimpse of both grammar-based model and statistical language model. These include lexical functional grammar, government and binding, Paninian grammar, and n -gram based model.

2.1 INTRODUCTION

Why and how do we model a language? This question has been discussed by linguists since 500 BC. Computational linguists also have to confront this question. It is obvious that our purpose is to understand and generate natural languages from a computational viewpoint. One approach can be to just take a language, try to understand every word and sentence of it, and then come to a conclusion. This approach has not succeeded as there are difficulties at each stage, which we will understand as we go through this book. An alternative approach is to study the grammar of various languages, compare them, and if possible, arrive at reasonable models that facilitate our understanding of the problem and designing of natural-language tools.

A model is a description of some complex entity or process. A language model is thus a description of language. Indeed, natural language is a complex entity and in order to process it through a computer-based program, we need to build a representation (model) of it. This is known

as *language modelling*. Language modelling can be viewed either as a problem of grammar inference or a problem of probability estimation. A grammar-based language model attempts to distinguish a grammatical sentence from a non-grammatical (ill-formed) one, whereas a probabilistic language model attempts to identify a sentence based on a probability measure, usually a maximum likelihood estimate. These two viewpoints have led to the following categorization of language modelling approaches.

Grammar-based language model

A grammar-based approach uses the grammar of a language to create its model. It attempts to represent the syntactic structure of language. Grammar consists of hand-coded rules defining the structure and ordering of various constituents appearing in a linguistic unit (phrase, sentence, etc.). For example, a sentence usually consists of noun phrase and a verb phrase. The grammar-based approach attempts to utilize this structure and also the relationships between these structures.

Statistical language modelling

The statistical approach creates a language model by training it from a corpus. In order to capture regularities of a language, the training corpus needs to be sufficiently large. Rosenfeld (1994) pointed out:

Statistical language modelling (SLM) is the attempt to capture regularities of natural language for the purpose of improving the performance of various natural language applications.

Statistical language modelling is one of the fundamental tasks in many NLP applications, including speech recognition, spelling correction, handwriting recognition, and machine translation. It has now found applications in information retrieval, text summarization, and question answering also. A number of statistical language models have been proposed in literature. The most popular of these are the n -gram models. We discuss this model in Section 2.3. The following section discusses various grammar-based models.

2.2 VARIOUS GRAMMAR-BASED LANGUAGE MODELS

Various computational grammars have been proposed and studied, e.g., transformational grammar (Chomsky 1957), lexical functional grammar (Kaplan and Bresnan 1982), government and binding (Chomsky 1981), generalized phrase structure grammar (Gazdar et al. 1985), dependency grammar, paninian grammar, and tree-adjoining grammar (Joshi 1985).

Issues in Paninian Grammar

The two problems challenging linguists are:

- (i) Computational implementation of PG, and
- (ii) Adaptation of PG to Indian, and other similar languages.

An approach to implementing PG has been discussed in Bharati, et al. (1995). This is a multilayered implementation. The approach is named 'Utsarga-Apvada' (default-exception), where rules are arranged in multiple layers in such a way that each layer consists of rules which are in exception to rules in the higher layer. Thus, as we go down the layer, more particular information is derived. Rules may be represented in the form of charts (such as Karaka chart and Lakshan chart).

However, many issues remain unresolved, specially in cases of shared Karak relations. Another difficulty arises when mapping between the Vibhakti (case markers and post-positions) and the semantic relation (with respect to verb) is not one to one. Two different Vibhakti can represent the same relation, or the same Vibhakti can represent different relations in different contexts. The strategy to disambiguate the various senses of words, or word groupings, are still the challenging issues.

As the system of rules is different in different languages, the framework requires adaptations to tackle various applications in various languages. Only some general features of PG framework has been described here.

2.3 STATISTICAL LANGUAGE MODEL

A statistical language model is a probability distribution $P(s)$ over all possible word sequences (or any other linguistic unit like words, sentences, paragraphs, documents, or spoken utterances). A number of statistical language models have been proposed in literature. The dominant approach in statistical language modelling is the n -gram model.

2.3.1 n -gram Model

As discussed earlier, the goal of a statistical language model is to estimate the probability (likelihood) of a sentence. This is achieved by decomposing sentence probability into a product of conditional probabilities using the chain rule as follows:

$$\begin{aligned}
 P(s) &= P(w_1, w_2, w_3, \dots, w_n) \\
 &= P(w_1) P(w_2/w_1) P(w_3/w_1 w_2) P(w_4/w_1 w_2 w_3) \dots \\
 &\quad P(w_n/w_1 w_2 \dots w_{n-1}) \\
 &= \prod_{i=1}^n P(w_i/h_i)
 \end{aligned}$$

where h_i is history of word w_i defined as

$$w_1 w_2 \dots w_{i-1}$$

So, in order to calculate sentence probability, we need to calculate the probability of a word, given the sequence of words preceding it. This is not a simple task. An n -gram model simplifies the task by approximating the probability of a word given all the previous words by the conditional probability given previous $n-1$ words only.

$$P(w_i/h_i) \approx P(w_i/w_{i-n+1} \dots w_{i-1})$$

Thus, an n -gram model calculates $P(w_i/h_i)$ by modelling language as Markov model of order $n-1$, i.e., by looking at previous $n-1$ words only. A model that limits the history to the previous one word only, is termed a bi-gram ($n=2$) model. Likewise, a model that conditions the probability of a word to the previous two words, is called a tri-gram ($n=3$) model. Using bi-gram and tri-gram estimate, the probability of a sentence can be calculated as:

$$P(s) \approx \prod_{i=1}^n P(w_i/w_{i-1})$$

$$\text{and } P(s) \approx \prod_{i=1}^n P(w_i/w_{i-2} \dots w_{i-1})$$

As an example, the bi-gram approximation of $P(\text{east/The Arabian knights are fairy tales of the})$ is

$$P(\text{east/the}),$$

whereas a tri-gram approximation is

$$P(\text{east/of the}).$$

A special word (pseudo word) $\langle s \rangle$ is introduced to mark the beginning of the sentence in bi-gram estimation. The probability of the first word in a sentence is conditioned on $\langle s \rangle$. Similarly, in tri-gram estimation, we introduce two pseudo-words $\langle s1 \rangle$ and $\langle s2 \rangle$.

Now, we discuss how to estimate these probabilities. This is done by training the n -gram model on the training corpus. We estimate n -gram parameters using the maximum likelihood estimation (MLE) technique, i.e., using relative frequencies. We count a particular n -gram in the training corpus and divide it by the sum of all n -grams that share the same prefix.

$$P(w_i/w_{i-n+1}, \dots, w_{i-1}) = \frac{C(w_{i-n+1}, \dots, w_{i-1}, w_i)}{\sum_w C(w_{i-n+1}, \dots, w_{i-1}, w)}$$

The sum of all n -grams that share first $n-1$ words is equal to the count of the common prefix $w_{i-n+1}, \dots, w_{i-1}$. So, we rewrite the previous expression as follows:

$$P(w_i/w_{i-n+1}, \dots, w_{i-1}) = \frac{C(w_{i-n+1}, \dots, w_{i-1}, w_i)}{C(w_{i-n+1}, \dots, w_{i-1})}$$

The model parameter we get using these estimates, maximizes the probability of the training set T given the model M , i.e., $P(T/M)$. The frequency with which a word occurs in a text may not be the same as in the training set; this model only provides the most likely solution.

A number of improvements have been suggested for the standard n -gram model. Before we discuss them, let us illustrate these ideas with the help of an example.

Example 2.9

Training set:

The Arabian Knights
 These are the fairy tales of the east
 The stories of the Arabian knights are translated in many languages

Bi-gram model:

$P(\text{the}/\langle s \rangle) = 0.67$	$P(\text{Arabian}/\text{the}) = 0.4$	$P(\text{knights}/\text{Arabian}) = 1.0$
$P(\text{are}/\text{these}) = 1.0$	$P(\text{the}/\text{are}) = 0.5$	$P(\text{fairy}/\text{the}) = 0.2$
$P(\text{tales}/\text{fairy}) = 1.0$	$P(\text{of}/\text{tales}) = 1.0$	$P(\text{the}/\text{of}) = 1.0$
$P(\text{east}/\text{the}) = 0.2$	$P(\text{stories}/\text{the}) = 0.2$	$P(\text{of}/\text{stories}) = 1.0$
$P(\text{are}/\text{knights}) = 1.0$	$P(\text{translated}/\text{are}) = 0.5$	$P(\text{in}/\text{translated}) = 1.0$
$P(\text{many}/\text{in}) = 1.0$		
$P(\text{languages}/\text{many}) = 1.0$		

Test sentence(s): The Arabian knights are the fairy tales of the east.

$$\begin{aligned}
 &P(\text{The}/\langle s \rangle) \times P(\text{Arabian}/\text{the}) \times P(\text{Knights}/\text{Arabian}) \times P(\text{are}/\text{knights}) \\
 &\times P(\text{the}/\text{are}) \times P(\text{fairy}/\text{the}) \times P(\text{tales}/\text{fairy}) \times P(\text{of}/\text{tales}) \times P(\text{the}/\text{of}) \\
 &\times P(\text{east}/\text{the}) \\
 &= 0.67 \times 0.5 \times 1.0 \times 1.0 \times 0.5 \times 0.2 \times 1.0 \times 1.0 \times 1.0 \times 0.2 \\
 &= 0.0067
 \end{aligned}$$

As each probability is necessarily less than 1, multiplying the probabilities might cause a numerical underflow, particularly in long sentences. To avoid this, calculations are made in log space, where a calculation corresponds to adding log of individual probabilities and taking antilog of the sum.

The n -gram model suffers from data sparseness problem. An n -gram that does not occur in the training data is assigned zero probability, that even a large corpus has several zero entries in its bi-gram matrix. This is because of the assumption that the probability of occurrence of word depends only on the preceding word (or preceding $n-1$ words) which is not true in general. There are several long distance dependencies in natural language sentences, which this model fails to capture. Goodman (2003) pointed out that 'there is rarely enough data to accurately estimate the parameters of a language model.'

A number of smoothing techniques have been developed to handle the data sparseness problem, the simplest of these being add-one smoothing. In the words of Jurafsky and Martin (2000):

Smoothing in general refers to the task of re-evaluating zero probability or low-probability n -grams and assigning them non-zero values.

The word 'smoothing' is used to denote these techniques because they tend to make distributions more uniform by moving the extreme probabilities towards the average.

2.3.2 Add-one Smoothing

This is the simplest smoothing technique. It adds a value of one to each n -gram frequency before normalizing them into probabilities. Thus, the conditional probability becomes:

$$P(w_i/w_{i-n+1}, \dots, w_{i-1}) = \frac{C(w_{i-n+1}, \dots, w_{i-1}, w_i)}{C(w_{i-n+1}, \dots, w_{i-1}) + V}$$

where V is the vocabulary size, i.e., size of the set of all the words being considered.

In general, add-one smoothing is not considered a good smoothing technique. It assigns the same probability to all missing n -grams, even though some of them could be more intuitively appealing than others. Gale and Church (1994) reported that variance of the counts produced by the add-one smoothing is worse than the unsmoothed MLE method. Another problem with this technique is that it shifts too much of the probability mass towards the unseen n -grams (n -grams with 0 probabilities) as their number is usually quite large. Good-Turing smoothing (Good 1953) attempts to improve the situation by looking at the number of n -grams with a high frequency in order to estimate the probability mass that needs to be assigned to missing or low-frequency n -grams.

2.3.3 Good-Turing Smoothing

Good-Turing smoothing (Good 1953) adjusts the frequency f of an n -gram using the count of n -grams having a frequency of occurrence $f+1$. It converts the frequency of an n -gram from f to f^* using the following expression:

$$f^* = (f + 1) \frac{n_{f+1}}{n_f}$$

where n_f is the number of n -grams that occur exactly f times in the training corpus. As an example, consider that the number of n -grams that occur 4 times is 25,108 and the number of n -grams that occur 5 times is 20,542. Then, the smoothed count for 5 will be

$$\frac{20542}{25108} \times 5 = 4.09$$

2.3.4 Caching Technique

Another improvement over basic n -gram model is caching. The frequency of n -gram is not uniform across the text segments or corpus. Certain words occur more frequently in certain segments (or documents) and rarely in others. For example, in this section, the frequency of the word ' n -gram' is high, whereas it occurs rarely in earlier sections. The basic n -gram model ignores this sort of variation of n -gram frequency. The cache model combines the most recent n -gram frequency with the standard n -gram model to improve its performance locally. The underlying assumption here is that the recently discovered words are more likely to be repeated.

A number of other smoothing techniques appear in literature. For these, readers are referred to Chen and Goodman (1998).

SUMMARY

The main topics covered in this chapter are as follows.

- Language modelling deals with providing a description of natural languages amenable to processing.
- There are two main approaches to language modelling: grammar-based language model and statistical language model.
- A grammar-based language model uses grammar to model language. A number of computational grammars have been proposed in literature. This chapter provides a discussion of models based on

acquire new members while closed word classes do not (or only infrequently do so). Nouns, verbs (except auxiliary verbs), adjectives, adverbs, and interjections are open word classes. Prepositions, auxiliary verbs, delimiters, conjunction, and particles are closed word classes.

Table 3.8 Part-of-speech example

NN	noun	student, chair, proof, mechanism
VB	verb	study, increase, produce
ADJ	adj	large, high, tall, few,
JJ	adverb	carefully, slowly, uniformly
IN	preposition	in, on, to, of
PRP	pronoun	I, me, they
DET	determiner	the, a, an, this, those

3.7 PART-OF-SPEECH TAGGING

Part-of-speech tagging is the process of assigning a part-of-speech (such as a noun, verb, pronoun, preposition, adverb, and adjective), to each word in a sentence. The input to a tagging algorithm is the sequence of words of a natural language sentence and specified tag sets (a finite list of part-of-speech tags). The output is a single best part-of-speech tag for each word. Many words may belong to more than one lexical category. For example, the English word 'book' can be a noun as in '*I am reading a good book*' or a verb as in '*The police booked the snatcher*'. The same is true for other languages. For example, the Hindi word '*sona*' may mean 'gold' (noun) or 'sleep' (verb). However, only one of the possible meanings is used at a time. In tagging, we try to determine the correct lexical category of a word in its context. No tagger is efficient enough to identify the correct lexical category of each word in a sentence in every case. The tag assigned by a tagger is the most likely for a particular use of word in a sentence.

The collection of tags used by a particular tagger is called a *tag set*. Most part-of-speech tag sets make use of the same basic categories, i.e., noun, verb, adjective, and prepositions. However, tag sets differ in how they define categories and how finely they divide words into categories. For example, both *eat* and *eats* might be tagged as a verb in one tag set, but assigned distinct tags in another tag set. In addition, most tag sets capture morpho-syntactic information such as singular/plural, number, gender, tense, etc. Consider the following sentences:

Zuha eats an apple daily.

Aman *ate* an apple yesterday.

They have *eaten* all the apples in the basket.

I like to *eat* guavas.

The word *eat* has a distinct grammatical form in each of these four sentences. *Eat* is the base form, *ate* its past tense, and the form *eat* requires a third person singular subject. Similarly, *eaten* is the past participle form and cannot occur in another grammatical context. It is required after have or has. Thus, the following sentences are ungrammatical:

I like to *eats* guava.

They *eaten* all the apples.

The number of tags used by different taggers varies substantially. Some use 20, while others use over 400 tags. The Penn Treebank tag set contains 45 tags while C7 uses 164. For a language like English, which is not morphologically rich, the C7 tagset is too big. The tagging process would yield too many mistagged words and the result would have to be manually corrected. Despite this, bigger tag sets have been used, e.g., TOSCA-ICE for the International Corpus of English with 270 tags (Garside 1997), or TESS with 200 tags. The larger the tag set, the greater the information captured about a linguistic context. However, the task of tagging becomes complicated and requires manual correction. A bigger tag set can be used for morphologically rich languages without introducing too many tagging errors. A tag set that uses just one tag to denote all the verbs will assign identical tags to all the forms of a verb. Although this coarse-grained distinction may be appropriate for some tasks, a fine-grained tag set captures more information. This is useful for tasks like syntactic pattern detection. The Penn Treebank tag set captures finer distinctions by assigning distinct tags to distinct grammatical forms of a verb, as summarized in Table 3.9. Tags assigned to the four different forms of the word *eat* according to this tag set is shown in Table 3.10.

Table 3.9 Tags from Penn Treebank tag set

VB	Verb, base form
	Subsumes imperatives, infinitives, and subjunctives
VBD	Verb, past tense
	Includes the conditional form of the verb <i>to be</i>
VBG	Verb, gerund, or present participle
VCN	Verb, past participle
VBP	Verb, non-3rd person singular present
VBS	Verb, 3rd person singular present

Table 3.10 Possible tags for the word to eat

eat	VB
ate	VBD
aten	VCN
eats	VBP

Here is an example of a tagged sentence:

Speech/NN sounds/NNS were/VBD sampled/VBN by/IN a/DT
microphone/NN.

The tag set used is Penn Treebank.

Another tagging possible for this sentence is as follows:

Speech/NN sounds/VBZ were/VBD sampled/VBN by/IN a/DT
microphone/NN.

It is easy to see that the second tagged sequence is not correct. It leads to semantic incoherence. We resolve the ambiguity using the context of the word. The context is also utilized by automatic taggers.

Part-of-speech tagging is an early stage of text processing in many NLP applications including speech synthesis, machine translation, information retrieval, and information extraction. In information retrieval, part-of-speech tagging can be used for indexing (for identifying useful tokens like nouns and phrases) and for disambiguating word senses. Tagging is not as complex as parsing. In tagging, a complete parse tree is not built; part-of-speech is assigned to words using contextual information.

Part-of-speech tagging methods fall under the three general categories.

- Rule-based (linguistic)
- Stochastic (data-driven)
- Hybrid

Rule-based taggers use hand-coded rules to assign tags to words. These rules use a lexicon to obtain a list of candidate tags and then use rules to discard incorrect tags.

Stochastic taggers have data-driven approaches in which frequency-based information is automatically derived from corpus and used to tag words. Stochastic taggers disambiguate words based on the probability that a word occurs with a particular tag. The simplest scheme is to assign the most frequent tag to each word. An early example of stochastic tagger was CLAWS (constituent likelihood automatic word-tagging system). CLAWS is the Stochastic equivalent of TAGGIT. Hidden Markov model (HMM) is the standard Stochastic tagger.

Hybrid taggers combine features of both these approaches. Like rule-based systems, they use rules to specify tags. Like stochastic systems, they use machine-learning to induce rules from a tagged training corpus automatically. The transformation-based tagger or Brill tagger is an example of the hybrid approach.

3.7.1 Rule-based Tagger

Most rule-based taggers have a two-stage architecture. The first stage is simply a dictionary look-up procedure, which returns a set of potential tags (parts-of-speech) and appropriate syntactic features for each word. The second stage uses a set of hand-coded rules to discard contextually illegitimate tags to get a single part-of-speech for each word. For example, consider the noun-verb ambiguity in the following sentence:

The show must go on.

The potential tags for the word *show* in this sentence is {VB, NN}. We resolve this ambiguity by using the following rule.

IF preceding word is determiner THEN eliminate VB tag.

This rule simply disallows verbs after a determiner. Using this rule the word *show* in the given sentence can only be noun.

In addition to contextual information, many taggers use morphological information to help in the disambiguation process. An example of a rule that make use of morphological information is:

IF word ends in -ing and preceding word is a verb THEN label it a verb (VB).

Capitalization information can be utilized in the tagging of unknown nouns. Rule-based taggers usually require supervised training. Instead, rules can be induced automatically. To induce rules untagged text is run through a tagger. The output is then manually corrected. The corrected text is then submitted to the tagger, which learns correction rules by comparing the two sets of data. This process may be repeated several times.

The earlier systems for automatic tagging were all rule-based. An example is TAGGIT (Greene and Rubin 1971), which was used for the initial tagging of the Brown corpus (Francis and Kucera 1982). This was also rule-based system. It used 3,300 disambiguation rules and was able to tag 77% of the words in the Brown corpus with their correct part-of-speech. The rest was done manually over several years. Yet another rule-based tagger is ENGTWOL (Voutilainen 1995).

Speed is an advantage of the rule-based tagger, and unlike stochastic taggers, they are deterministic. One of the arguments against them is the skill and effort required in writing disambiguation rules. However, stochastic taggers also require manual work if good performance is to be achieved. In the rule-based system, time is spent in writing a rule-set. For stochastic taggers, time is spent developing restrictions on transitions and emissions to improve tagger performance. Another disadvantage of the rule-based tagger is that it is usable for only one language. Using it for another one requires a rewrite of most of the program; unlike a probabilistic tagger which would be usable with only slight changes and new training.

3.7.2 Stochastic Tagger

The standard stochastic tagger algorithm is the HMM tagger. A Markov model applies the simplifying assumption that the probability of a chain of symbols can be approximated in terms of its parts or n -grams. The simplest n -gram model is the unigram model, which assigns the most likely tag (part-of-speech) to each token.

The unigram model needs to be trained using a tagged training corpus before it can be used to tag data. The most likely statistics are gathered over the corpus and used for tagging. The context used by the unigram tagger is the text of the word itself. For example, it will assign the tag JJ for each occurrence of *fast*, since *fast* is used as an adjective more frequently than it is used as a noun, verb, or adverb. This results in incorrect tagging in each of the following sentences:

She had a *fast*. (3.1)

Muslims *fast* during Ramadan. (3.2)

Those who were injured in the accident need to be helped *fast*. (3.3)

In the first sentence, *fast* is used as a noun. In the second, it is a verb, and in the third, an adverb.

We would expect more accurate predictions if we took more context into account when making a tagging decision. A bi-gram tagger uses the current word and the tag of the previous word in the tagging process. As the tag sequence "DT NN" is more likely than the tag sequence "DT JJ", a bi-gram model will assign a correct tag to the word *fast* in sentence (3.1). Similarly, it is more likely that an adverb (rather than a noun or an adjective) follows a verb. Hence, in sentence (3.3), the tag assigned to *fast* will be RB.

In general, an n -gram model considers the current word and the tag of the previous $n-1$ words in assigning a tag to a word. The context considered by a tri-gram model is shown in Figure 3.15. The area shaded in gray represents the context.

Tokens	w_{n-2}	w_{n-1}	w_n	w_{n+1}
tags	t_{n-2}	t_{n-1}	t_n	t_{n+1}

Figure 3.15 Context used by a tri-gram tagger

So far, we have considered how a tag is assigned to a word given the previous tag(s). However, the objective of a tagger is to assign a tag sequence to a given sentence. We now discuss how the HMM tagger assigns the most likely tag sequence to a given sentence. We call this the HMM because it uses two layers of states: a visible layer corresponding to the input words, and a hidden layer learnt by the system corresponding to the tags. While tagging the input data, we only observe the words—the tags (states) are hidden. States of the model are visible in training, not during the tagging task.

As discussed earlier, the HMM makes use of lexical and bi-gram probabilities estimated over a tagged training corpus in order to compute the most likely tag sequence for each sentence. One way to store this statistical information is to build a probability matrix. The probability matrix contains both the probability that an individual word belongs to a word class as well as the n -gram analysis, e.g., for a bi-gram model, the probability that a word of class X follows a word of class Y. This matrix is then used to drive the HMM tagger while tagging an unknown text.

We now return to the original problem. Given a sequence of words (sentence), the objective is to find the most probable tag sequence for the sentence.

Let W be the sequence of words.

$$W = w_1, w_2, \dots, w_n$$

The task is to find the tag sequence

$$T = t_1, t_2, \dots, t_n$$

which maximizes $P(T|W)$, i.e.,

$$T' = \operatorname{argmax}_T P(T|W)$$

Applying Bayes Rule, $P(T|W)$ can be estimated using the expression

$$P(T|W) = P(W|T) \cdot P(T)/P(W)$$

As the probability of the word sequence, $P(W)$, remains the same for each tag sequence, we can drop it. The expression for the most likely tag sequence becomes:

$$T' = \operatorname{argmax}_T P(W|T) * P(T)$$

Using the Markov assumption, the probability of a tag sequence can be estimated as the product of the probability of its constituent n -grams, i.e.,

$$P(T) = P(t_1) * P(t_2|t_1) * P(t_3|t_1 t_2) \dots * P(t_n|t_1 \dots t_{n-1})$$

$P(W/T)$ is the probability of seeing a word sequence, given a tag sequence. For example, it is asking the probability of seeing 'The egg is rotten' given 'DT NNP VB JJ'. We make the following two assumptions:

- The words are independent of each other.
- The probability of a word is dependent only on its tag.

Using these assumptions, we obtain

$$P(W/T) = P(w_1/t_1) * P(w_2/t_2) \dots P(w_i/t_i) * \dots P(w_n/t_n)$$

$$\text{i.e., } P(W/T) \approx \prod_{i=1}^n P(w_i|t_i)$$

$$\text{So, } P(W/T) * P(T) = \prod_{i=1}^n P(w_i|t_i)$$

$$\times P(t_1) \times P(t_2|t_1) * P(t_3|t_1 t_2) \times \dots \times P(t_n|t_1 \dots t_{n-1})$$

Approximating the tag history using only the two previous tags, the transition probability, $P(T)$, becomes

$$P(T) = P(t_1) \times P(t_2|t_1) \times P(t_3|t_1 t_2) \times \dots \times P(t_n|t_{n-2} t_{n-1})$$

Hence, $P(T/W)$ can be estimated as

$$\begin{aligned} P(W/T) * P(T) &= \prod_{i=1}^n P(w_i|t_i) \\ &\times P(t_1) \times P(t_2|t_1) * P(t_3|t_2 t_1) \times \dots \times P(t_n|t_{n-2} t_{n-1}) \\ &= \prod_{i=1}^n P(w_i|t_i) \times P(t_1) \times P(t_2|t_1) \times \prod_{i=3}^n P(t_i|t_{i-2} t_{i-1}) \end{aligned}$$

We estimate these probabilities from relative frequencies via Maximum Likelihood Estimation.

$$P(t_i|t_{i-2} t_{i-1}) = \frac{c(t_{i-2}, t_{i-1}, t_i)}{c(t_{i-2}, t_{i-1})}$$

$$P(w_i|t_i) = \frac{c(w_i, t_i)}{c(t_i)}$$

where $c(t_{i-2}, t_{i-1}, t_i)$ is the number of occurrences of t_i followed by $t_{i-2} t_{i-1}$.

Stochastic models have the advantage of being accurate and language independent. Most stochastic taggers have an accuracy of 96-97%. This accuracy seems to be quite high but it should be noted that this is measured as a percentage of words. An accuracy of 96% means that for a sentence containing 20 words, the error rate per sentence will be $1 - 0.96^{20} = 56\%$. This corresponds to approximately one word per sentence.

One of the drawbacks of stochastic taggers is that they require a manually tagged corpus for training. Kupiec (1992), Cutting *et al.* (1992) and others have demonstrated that the HMM tagger can be trained from unannotated text. This makes it possible to use the model for language in which a manually tagged corpus is not available. However, a tagger trained on a hand-coded corpus performs better than one trained on an unannotated text. In order to achieve good performance a tagged corpus is required.

We now consider an example demonstrating how the probability of a particular part-of-speech sequence for a given sentence can be computed.

Example 3.3 Consider the sentence

The bird can fly.

and the tag sequence

DT NNP MD VB

Using bi-gram approximation, the probability

$$P \begin{pmatrix} \text{DT} & \text{NNP} & \text{MD} & \text{VB} \\ | & | & | & | \\ \text{The} & \text{bird} & \text{can} & \text{fly} \end{pmatrix}$$

can be computed as

$$= P(\text{DT}) \times P(\text{NNP}|\text{DT}) \times P(\text{MD}|\text{NNP}) \times P(\text{VB}|\text{MD}) \\ \times P(\text{the}/\text{DT}) \times P(\text{bird}|\text{NNP}) \times P(\text{can}|\text{MD}) \times P(\text{fly}|\text{VB})$$

3.7.3 Hybrid Taggers

Hybrid approaches to tagging combine the features of both the rule based and stochastic approaches. They use rules to assign tags to words. Like the stochastic taggers, this is a machine learning technique and rules are automatically induced from the data. Transformation-based learning (TBL) of tags, also known as Brill tagging, is an example of hybrid approach. TBL is a machine learning method introduced by E. Brill (1995). Transformation-based error-driven learning has been applied to a number of natural language problems, including part-of-speech tagging (Brill 1994), speech generation, and syntactic parsing (Brill 1993, 1994, Huang *et al.* 1994).

Stochastic models have the advantage of being accurate and language independent. Most stochastic taggers have an accuracy of 96–97%. This accuracy seems to be quite high but it should be noted that this is measured as a percentage of words. An accuracy of 96% means that for a sentence containing 20 words, the error rate per sentence will be $1 - 0.96^{20} = 5$. This corresponds to approximately one word per sentence.

One of the drawbacks of stochastic taggers is that they require a manually tagged corpus for training. Kupiec (1992), Cutting *et al.* (1990) and others have demonstrated that the HMM tagger can be trained from unannotated text. This makes it possible to use the model for language in which a manually tagged corpus is not available. However, a tagger trained on a hand-coded corpus performs better than one trained on unannotated text. In order to achieve good performance a tagged corpus is required.

We now consider an example demonstrating how the probability of a particular part-of-speech sequence for a given sentence can be computed.

Example 3.3 Consider the sentence

The bird can fly.

and the tag sequence

DT NNP MD VB

Using bi-gram approximation, the probability

$$P \begin{pmatrix} \text{DT} & \text{NNP} & \text{MD} & \text{VB} \\ | & | & | & | \\ \text{The} & \text{bird} & \text{can} & \text{fly} \end{pmatrix}$$

can be computed as

$$= P(\text{DT}) \times P(\text{NNP}|\text{DT}) \times P(\text{MD}|\text{NNP}) \times P(\text{VB}|\text{MD}) \\ \times P(\text{the}/\text{DT}) \times P(\text{bird}/\text{NNP}) \times P(\text{can}/\text{MD}) \times P(\text{fly}/\text{VB})$$

3.7.3 Hybrid Taggers

Hybrid approaches to tagging combine the features of both the rule-based and stochastic approaches. They use rules to assign tags to words. Like the stochastic taggers, this is a machine learning technique and rules are automatically induced from the data. Transformation-based learning (TBL) of tags, also known as Brill tagging, is an example of hybrid approach. TBL is a machine learning method introduced by E. Brill (1995). Transformation-based error-driven learning has been applied to a number of natural language problems, including part-of-speech tagging (1994), speech generation, and syntactic parsing (Brill 1993, 1994, Huang *et al.* 1994).

Researchers have proposed a number of parsing methods for natural language sentences. It is not possible to discuss all of them in a chapter, so we focus on a few widely-known ones. Most text books use only English for their discussions. We differ by including Indian languages as well. In addition to an introduction to grammar formalism, this chapter also provides an introduction to Hindi grammar.

Two important ideas in natural language are those of constituency and word order. Constituency is about how words are grouped together and how we know that they are really grouping together. Word order is about how, within a constituent, words are ordered with respect to one another, and also how constituents are ordered with respect to one another.

A widely used mathematical system for modelling constituent structure in natural language is context-free grammar (CFG) also known as phrase structure grammar. We begin our discussion with an overview of CFG in Section 4.2. In Section 4.3, we discuss various types of phrases and phrase structure rules. We then introduce feature structures in Section 4.4, to capture certain properties of grammatical categories which cannot be handled efficiently using CFG. We discuss probabilistic grammar and CYK parser based on this in Section 4.5. The CFG is basically a positional grammar and is not suitable for Indian languages, which are free word order languages. We provide a brief overview of Paninian grammar (PG), which is suitable for modelling free word order languages, in Section 4.6. We also discuss a parsing framework proposed by Bharti and Sangal (1990) for Indian languages based on this grammar. Finally, we provide a brief summary of the chapter.

4.2 CONTEXT-FREE GRAMMAR

Context-free grammar (CFG) was first defined for natural language by Chomsky (1957) and used for the Algol programming language by Backus (1959) and Naur (1960). A CFG (also called phrase-structure grammar) consists of four components:

1. A set of non-terminal symbols, N
2. A set of terminal symbols, T
3. A designated start symbol, S , that is one of the symbols from N .
4. A set of productions, P , of the form:

$$A \rightarrow \alpha$$

where $A \in N$ and α is a string consisting of terminal and non-terminal symbols. The rule $A \rightarrow \alpha$ says that constituent A can be rewritten as α . This is also called the phrase structure rule. It specifies which elements

(or constituents) can occur in a phrase and in what order. For example, the rule $S \rightarrow NP VP$ states that S consists of NP followed by VP , i.e., a sentence consists of a noun phrase followed by a verb phrase.

A language is usually defined through the concept of derivation. The basic operation is that of rewriting a symbol appearing on the left hand side of production by its right hand side. A CFG can be used to generate a sentence or to assign a structure to a given sentence. When used as a generator, the arrows in the production rule may be read as 'rewrite the symbol on the left with symbols on the right'. Consider the toy grammar shown in Figure 4.1. The symbol S can be rewritten as $NP VP$ using Rule 1, then using rules R2 and R4, NP and VP are rewritten as N and $V NP$ respectively. NP is then rewritten as $Det N$ (R3). Finally, using rules R6 and R7, we get the sentence:

Hena reads a book.

(4.1)

We say that the sentence (4.1) can be derived from S . The representation of this derivation is shown in Figure 4.1. Sometimes, a more compact bracketed notation is used to represent a parse tree. The parse tree in Figure 4.1 can be represented using this notation as follows:

$[S [NP [N Hena]] [VP [V reads] [NP [Det a] [N book]]]]$

The set of all the strings containing terminal symbols which can be derived from the start symbol of the grammar, defines the language generated by that grammar. The parse tree shown in Figure 4.1 essentially represents a mapping of a string to its parse tree. This mapping process is called parsing. We will come back to this issue in Section 4.4.

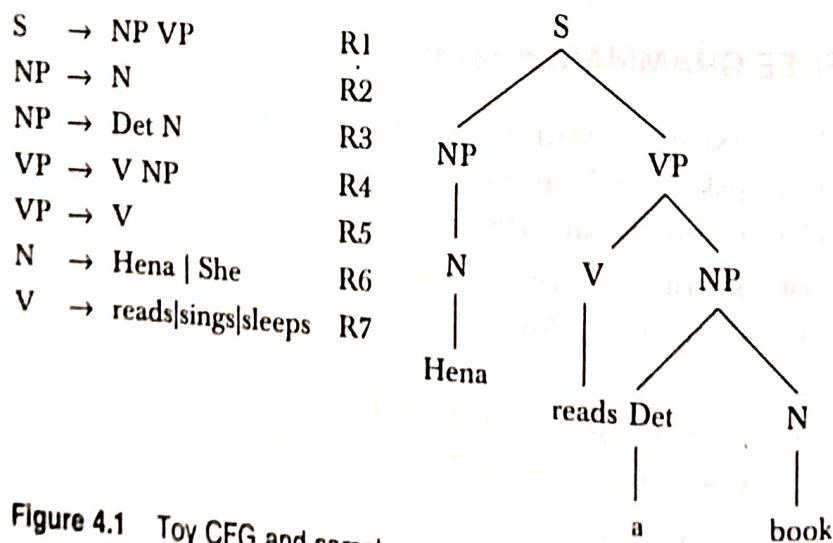


Figure 4.1 Toy CFG and sample parse tree

Garden pathing is another phenomenon related to syntactic parsing. It refers to the process of constructing a parse by exploring the parse tree along different paths, one after the other till, eventually, the right one is found. The popular example given for garden pathing is the sentence

The horse ran past the barn fell. (4.6)

In the first attempt, most of us come up with a parse corresponding to the sentence *The horse ran past the barn*, leaving no possibility for the word *fell* to be incrementally added in the sentence. In order to complete the parse of the sentence, we have to backtrack.

Finding the right parse can be viewed as a search process. The search finds all trees whose root is the start symbol *S* and whose leaves cover exactly the word in the input. The search space in this conception corresponds to all possible parse trees defined by the grammar. The following constraints guide the search process.

1. *Input*: The first constraint comes from the words in the input sentence. A valid parse is one that covers all the words in a sentence. Hence, these words must constitute the leaves of the final parse tree.
2. *Grammar*: The second kind of constraint comes from the grammar. The root of the final parse tree must be the start symbol of the grammar.

These two constraints give rise to the two most widely used search strategies by parsers, namely, top-down or goal-directed search and bottom-up or data-directed search.

4.4.1 Top-down Parsing

As the name suggests, top-down parsing starts its search from the root node *S* and works downwards towards the leaves. The underlying assumption here is that the input can be derived from the designated start symbol, *S*, of the grammar. The next step is to find all sub-trees which can start with *S*. To generate the sub-trees of the second-level search, we expand the root node using all the grammar rules with *S* on their left hand side. Likewise, each non-terminal symbol in the resulting sub-trees is expanded next using the grammar rules having a matching non-terminal symbol on their left hand side. The right hand side of the grammar rules provide the nodes to be generated, which are then expanded recursively. As the expansion continues, the tree grows downward and eventually reaches a state where the bottom of the tree consist only of part-of-speech categories. At this point, all trees whose leaves do not match words in the input sentence are rejected, leaving only trees that represent successful

parses. A successful parse corresponds to a tree which matches exactly with the words in the input sentence.

Table 4.2 Sample grammar

$S \rightarrow NP VP$	$VP \rightarrow Verb NP$
$S \rightarrow VP$	$VP \rightarrow Verb$
$NP \rightarrow Det Nominal$	$PP \rightarrow Preposition NP$
$NP \rightarrow Noun$	$Det \rightarrow this \mid that \mid a \mid the$
$NP \rightarrow Det Noun PP$	$Verb \rightarrow sleeps \mid sings \mid open \mid saw \mid paint$
$Nominal \rightarrow Noun$	$Preposition \rightarrow from \mid with \mid on \mid to$
$Nominal \rightarrow Noun Nominal$	$Pronoun \rightarrow she \mid he \mid they$

Consider the grammar shown in Table 4.2 and the sentence
Paint the door.

A top-down search begins with the start symbol of the grammar. Thus the first level (ply) search tree consists of a single node labelled S. The grammar in Table 4.2 has two rules with S on their left hand side. Then

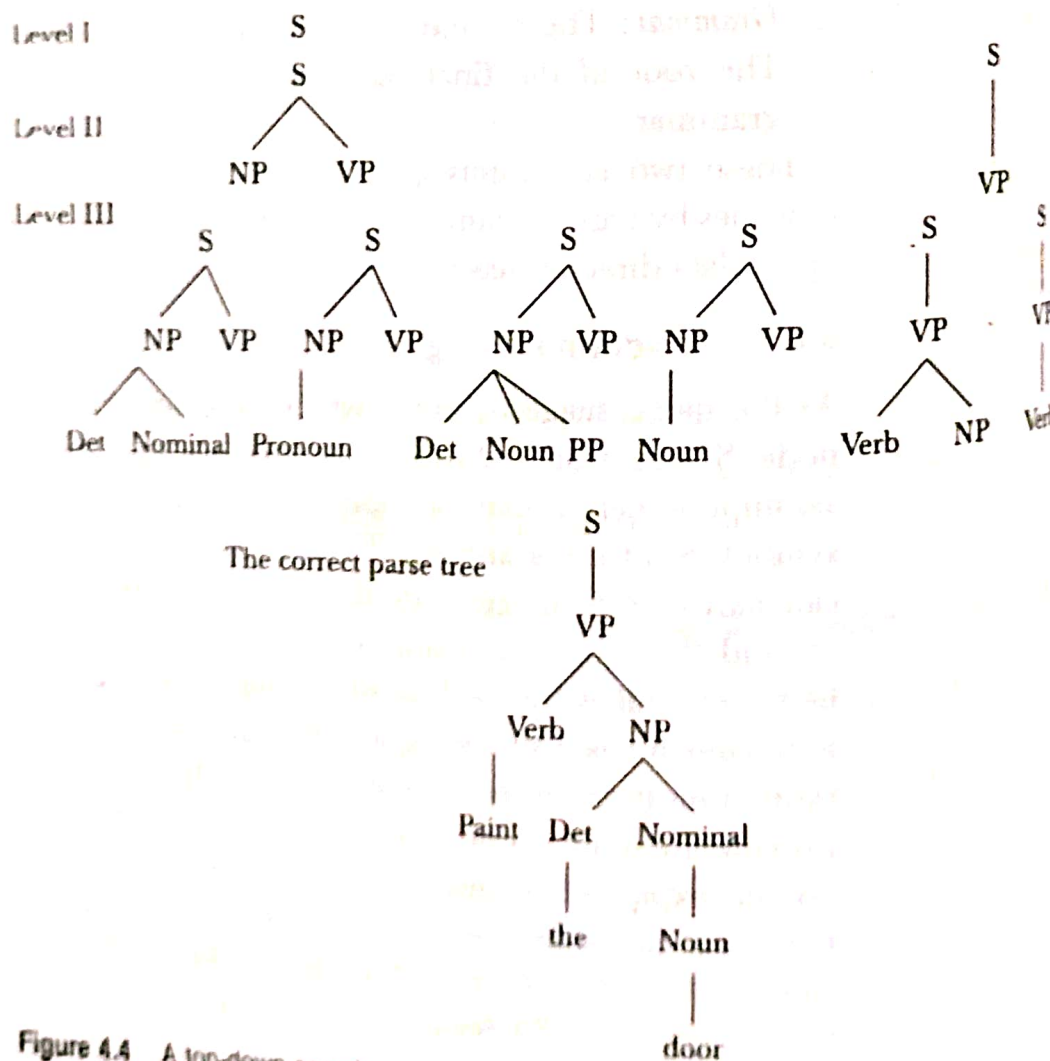


Figure 4.4 A top-down search space

rules are used to expand the tree, which gives us two partial trees at the second level search, as shown in Figure 4.4. The third level is generated by expanding the non-terminal at the bottom of the search tree in the previous ply. Due to space constraints, only the expansion corresponding to the left-most non-terminals has been shown in the figure. The subsequent steps in the parse are left, as an exercise, to the readers. The correct parse tree shown in Figure 4.4 is obtained by expanding the fifth parse tree of the third level.

4.4.2 Bottom-up Parsing

A bottom-up parser starts with the words in the input sentence and attempts to construct a parse tree in an upward direction towards the root. At each step, the parser looks for rules in the grammar where the right hand side matches some of the portions in the parse tree constructed so far, and reduces it using the left hand side of the production. The parse is considered successful if the parser reduces the tree to the start symbol of the grammar. Figure 4.5 shows some steps carried out by the bottom-up parser for sentence (4.7).

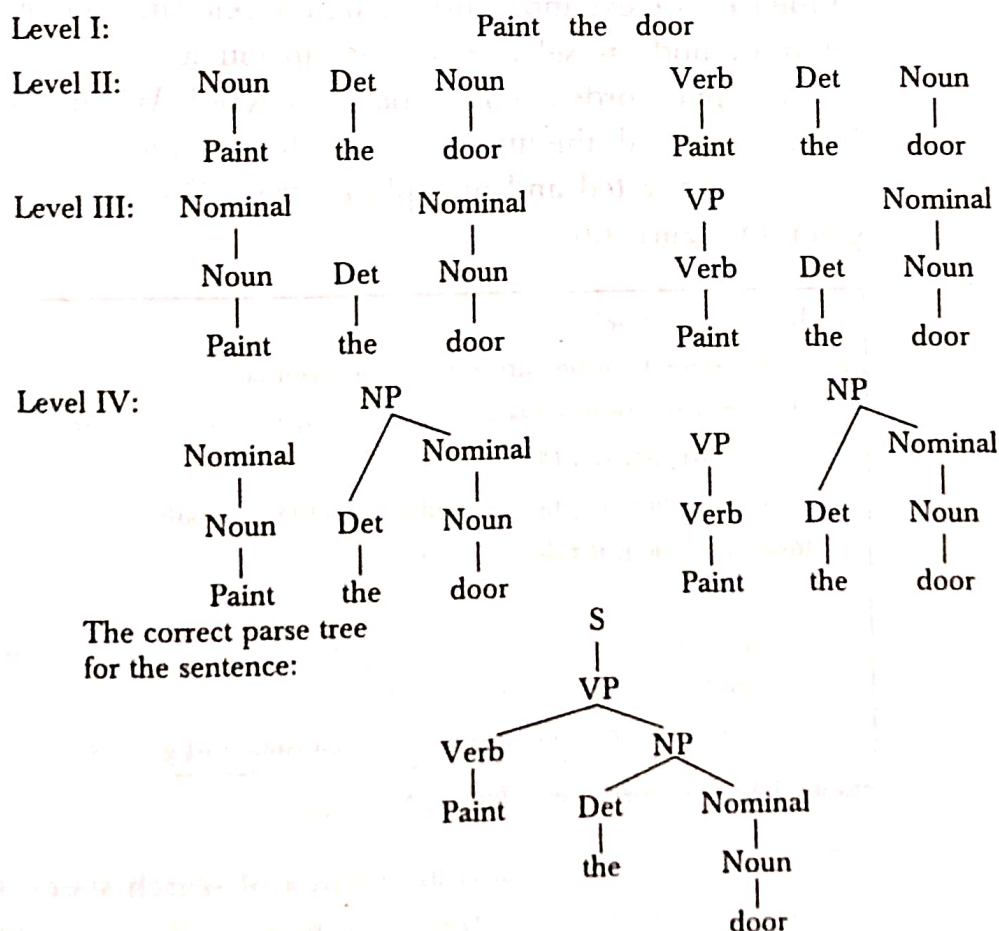


Figure 4.5 A bottom-up search space for sentence (4.7)

Each of these parsing strategies has its advantages and disadvantages. As the top-down search starts generating trees with the start symbol of the grammar, it never wastes time exploring a tree leading to a different root. However, it wastes considerable time exploring S trees that eventually result in words that are inconsistent with the input. This is because a top-down parser generates trees before seeing the input. On the other hand, a bottom-up parser never explores a tree that does not match the input. However, it wastes time generating trees that have no chance of leading to an S-rooted tree. The left branch of the search space in Figure 4.5 that explores a sub-tree assuming *paint* as a noun, is an example of wasted effort. We now present a basic search strategy that uses the top-down method to generate trees and augments it with bottom-up constraints to filter bad parses.

4.4.3 A Basic Top-down Parser

The approach presented here is essentially a depth first, left to right search. The depth first approach expands the search space incrementally by one state at a time. At each step, the left-most unexpanded leaf nodes of the tree are expanded first using the relevant rule of the grammar. The left-most node is selected for expansion as it determines the order in which input words needs to be considered. When a state arrives that is inconsistent with the input, the search continues by returning to the most recently generated and unexplored tree. The steps of the algorithm are given in Figure 4.6.

1. Initialize agenda
2. Pick a state, let it be `curr_state`, from agenda
3. If (`curr_state`) represents a successful parse then return parse tree
 - else if `curr_state` is a POS then
 - if category of `curr_state` is a subset of POS associated with `curr_word`
 - then apply lexical rules to current state
 - else reject
 - else generate new states by applying grammar rules and push them into agenda
4. If (agenda is empty) then return failure
 - else select a node from agenda for expansion and go to step 3.

Figure 4.6 Top-down, depth-first parsing algorithm

The algorithm maintains an agenda of search states. Each search state consists of partial trees and a pointer to the next input word in the sentence. The algorithm starts with the state at the front of the agenda

and generates a set of new states by applying grammar rule to the left-most unexpanded node of the tree associated with it. The newly generated states are put on the front of the agenda in the order defined by the textual order of the grammar rules used to create them. The process continues until either a successful parse tree is discovered or the agenda is empty, indicating a failure.

Figure 4.7 shows the trace of the algorithm on the sentence, *Open the door*. The algorithm starts with the node S and input word *Open*. It first expands S using the grammar rule $S \rightarrow NP VP$. It then expands the left-most unexpanded non-terminal NP using the rule $NP \rightarrow Det Nominal$. But the word *Open* cannot be derived from Det. Hence, the parser eliminates the rule and tries the second alternative, i.e., $NP \rightarrow noun$, which again leads to a failure. The next search space on the agenda corresponds to the $S \rightarrow VP$ rule. The expansion of VP using the rule $VP \rightarrow Verb NP$, successfully matches the first input words. The algorithm proceeds in a depth-first, left-to-right manner, to match the rest of the input words.

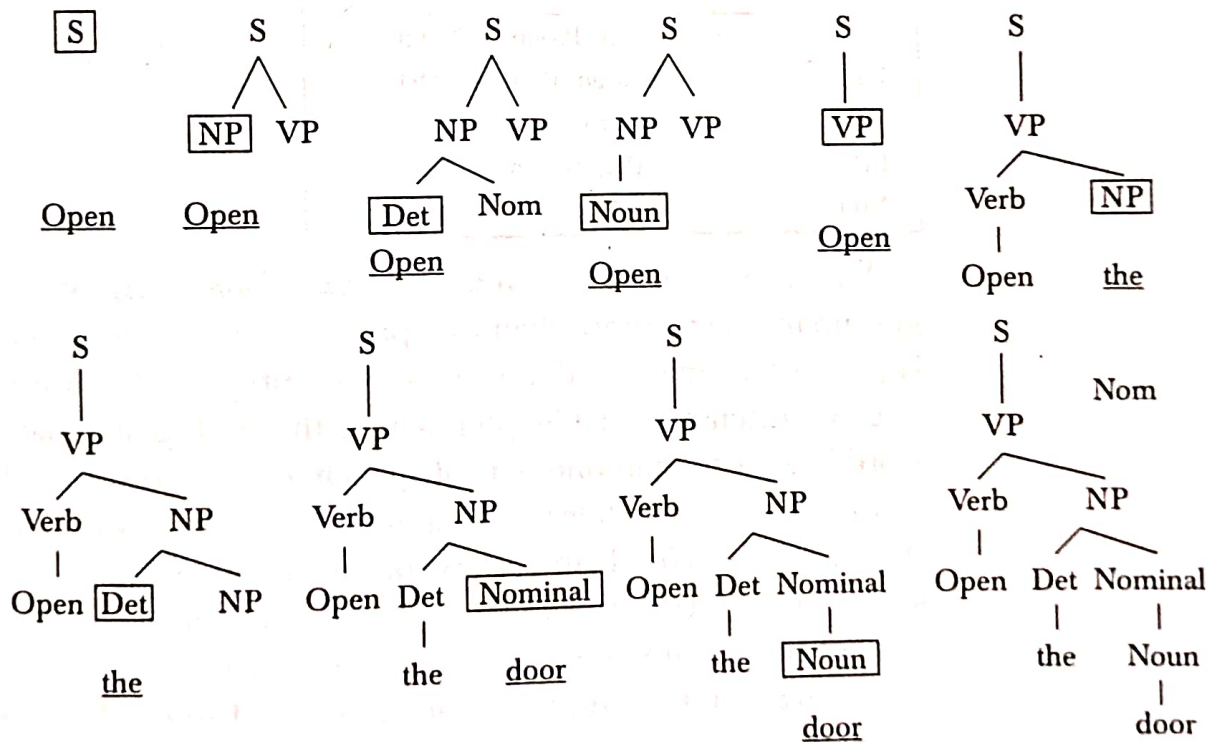


Figure 4.7 A derivation using top-down, depth-first algorithm

In any successful parse, the current input word must match the first word in the derivation of the node that is being expanded. This information can be utilized in eliminating spurious parses. A grammar rule that cannot lead to the input word as the first word along the left side of a derivation, should not be considered for expansion. The first word along the left side

of the derivation is called the left corner of the tree. Using the left corner notion, we see that in our example, only the rule $S \rightarrow VP$ is applicable, the word *Open* cannot be a left corner of the NP. In order to utilize the filter, we create a table containing a list of all the valid left corner categories for each non-terminal of the grammar. While selecting a rule for expansion the table is consulted to see if the non-terminal associated with the rule has a part-of-speech associated with the current input. If not, then the rule is not considered. The left corner table for grammar is shown in Table 4.3.

The top-down, depth-first, left-to-right search algorithm suffers from certain disadvantages. The first is that of left recursion, which causes the search to get stuck in an infinite loop. This problem arises if the grammar is left recursive that, is, it contains a non-terminal A , which derives, in one or more steps, a string beginning with the same non-terminal, i.e., $A \Rightarrow A\beta$ for some β .

Table 4.3 Left corner for each grammar category

Category	Left Corners
S	Det, Pronoun, Noun, Verb
NP	Noun, Pronoun, Det
VP	Verb
PP	Preposition
Nominal	Noun

The second problem is that of *structural ambiguity*, which occurs when a grammar assigns more than one parse to a sentence. Structural ambiguity is just one of many different types of ambiguities that arise in NLP. The one we discussed in Chapter 3 was the ambiguity that occurs when a word has more than one part-of-speech associated with it. In the following chapter, we review these ambiguities and discuss various disambiguation strategies. Structural ambiguity occurs in many forms, e.g., attachment ambiguity and coordination ambiguity. A sentence has *attachment ambiguity* if a constituent fits more than one position in a parse tree. For example, according to the grammar summarized in Table 4.1, there are two ways of generating the prepositional phrase, *with a long stick*, in the sentence *The girl plucked the flower with a long stick*. It can be generated from the verb phrase, as in the parse tree shown in Figure 4.2, or from the noun phrase, as in the parse tree shown in Figure 4.8. The first parse leads to the interpretation that the stick is used to pluck the flower, whereas the second parse gives the interpretation that the flower being plucked has a long stick.

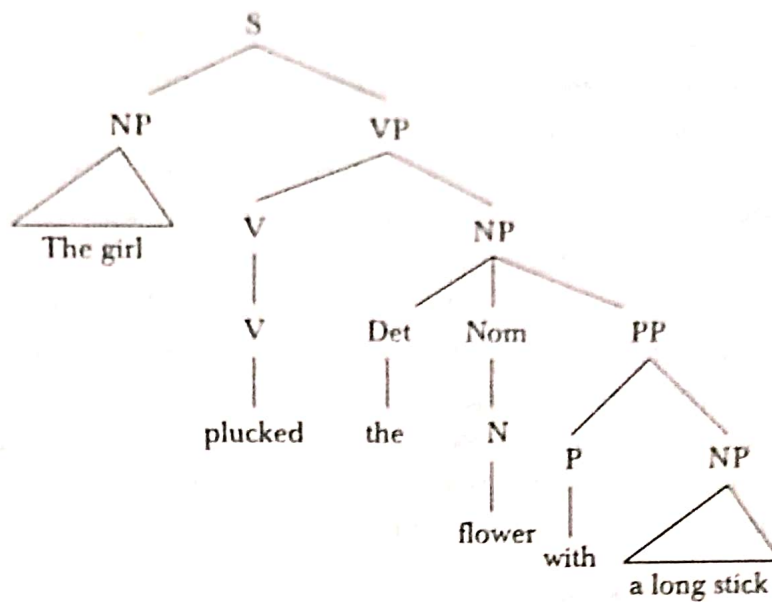


Figure 4.8 PP-attachment ambiguity

Coordination ambiguity occurs when it is not clear which phrases are being combined with a conjunction like *and*. For example, the phrase *beautiful hair and eyes* may have the structure [*beautiful hair*] and [*eyes*] or [*beautiful hair*] and [*beautiful eyes*]. Identifying the correct parse from a number of possible parses is known as disambiguation. A parser may utilize statistical and semantic knowledge to disambiguate the parse tree, or it may return all possible parses and leave the disambiguation for subsequent processing. The basic top-down parser we discussed, returns the first successful parse without exploring other possibilities. It needs to be modified to return all possible parses.

A sentence may have *local ambiguity* resulting in inefficient parsing. Local ambiguity occurs when certain parts of a sentence are ambiguous. For example, the sentence *Paint the door* is unambiguous, but during parsing it is not known whether the first word *Paint* is a verb or a noun. Hence, the parser makes a few incorrect expansions before discovering that *Paint* is a verb. Thus, it must use backtracking, or parallelism, to consider both parses.

Yet another problem associated with our basic top-down strategy is that of repeated parsing. The parser often builds valid trees for portions of the input that it discards during backtracking. These have to be rebuilt during subsequent steps in the parse. If we could avoid this extra effort, we would have more efficient parsers.

Dynamic programming algorithms can solve these problems. These algorithms construct a table containing solutions to sub-problems, which, if solved, will solve the whole problem. In parsing, a dynamic programming

algorithm builds a table containing sub-trees for each and every constituent appearing in the input. The efficiency comes from the fact that once these sub-trees are discovered, they are stored and later consulted by all parses attempting to expand that constituent. This solves the reparsing and ambiguity problem. There are three widely known dynamic parsers—the Cocke-Younger-Kasami (CYK) algorithm, the Graham-Harrison-Ruzzo (GHR) algorithm, and the Earley algorithm. We now present the Earley and CYK parsing algorithms to illustrate top-down and a bottom-up dynamic programming algorithms respectively.

Probabilistic grammar can also be used to disambiguate parse trees. We discuss probabilistic parsing and how they can be used to identify a likely parse, in the next section.

4.4.4 Earley Parser

The Earley parser implements an efficient parallel top-down search using dynamic programming. It builds a table of sub-trees for each of the constituents in the input. This way, the algorithm eliminates the repetitive parse of a constituent which arises from backtracking, and successfully reduces the exponential-time problem to polynomial time. The Earley parser can handle recursive rules such as $A \rightarrow AC$ without getting into an infinite loop.

The most important component of this algorithm is the Earley chart that has $n+1$ entries, where n is the number of words in the input. The chart contains a set of states for each word position in the sentence. The algorithm makes a left to right scan of input to fill the elements in this chart. It builds a set of states, one for each position in the input string (starting from 0), that describe the condition of the recognition process at that point in the scan. The states in each entry provide the following information.

1. A sub-tree corresponding to a grammar rule.
2. Information about the progress made in completing the sub-tree.
3. Position of the sub-tree with respect to input.

A state is represented as a dotted rule and a pair of numbers representing starting position and the position of dot. This representation takes the form

$$A \rightarrow X_1 \dots \bullet C \dots X_m [i, j]$$

where the dot ($\bullet$) represents the position in the rule's right hand side, and the two numbers (i and j) represent where the state begins and where the dot lies. A dot at the right end of the rule represents a successful parse of the associated non-terminal.

Earley Parsing

Input: Sentence and the Grammar

Output: Chart

```

chart[0]  $\leftarrow S' \rightarrow S, [0,0]$ 
 $n \leftarrow \text{length}(\text{sentence})$  // number of words in the sentence
for  $i = 0$  to  $n$  do
    for each state in chart[i] do
        if (incomplete (state) and next category is not a part of speech) then
            predictor (state)
        else if (incomplete (state) and next category is a part of speech)
            scanner (state)
        else
            completer (state)
        end-if
    end-if
end for
end for
return

Procedure predictor ( $A \rightarrow X_1 \dots \bullet B \dots X_m, [i, j]$ )
for each rule ( $B \rightarrow \alpha$ ) in  $G$  do
insert the state  $B \rightarrow \bullet \alpha, [j, j]$  to chart [j]
End

Procedure scanner ( $A \rightarrow X_1 \dots \bullet B \dots X_m, [i, j]$ )
If  $B$  is one of the part of speech associated with word[j] then
Insert the state  $B \rightarrow \text{word}[j] \bullet, [j, j+1]$  to chart [j+1]
End

Procedure Completer ( $A \rightarrow X_1 \dots \bullet, [j, k]$ )
for each  $B \rightarrow X_1 \dots \bullet A \dots, [i, j]$  in chart[j] do
insert the state  $B \rightarrow X_1 \dots A \bullet \dots, [i, k]$  to chart[k]
End

```

Figure 4.9 The Earley Parsing Algorithm

The algorithm uses three operations to process states in the chart. These are:

- Predictor
- Scanner
- Completer

The algorithm sequentially constructs the sets for each of the $n+1$ chart entries. Chart [0] is initialized with a dummy state $S' \rightarrow \bullet S, [0,0]$. At each step one of the three operations are applicable depending on the state.

Application of these operators result in addition of new states to either the current or the next set of states. The presence of a state $S \rightarrow \alpha, [i, j]$ indicates a successful parse. The algorithm is shown in Figure 4.9.

We now explain the function of the three operators.

Predictor

As the name suggests, the predictor generates new states representing potential expansion of the non-terminal in the left-most derivation. The predictor is applied to every state that has a non-terminal to the right of the dot, when the category of that non-terminal is different from the part-of-speech. The application of this operator results in the creation of as many new states as there are grammar rules for the non-terminal. These new states are placed into the same chart entry as the generating state. Their start and end positions are at the point where the generating state ends. If

$$A \rightarrow X_1 \dots \bullet B \dots X_m [i, j]$$

Then for every rule of the form $B \rightarrow \alpha$, the operation adds to chart $[j]$ the state

$$B \rightarrow \bullet \alpha, [j, j]$$

For example, when the generating state is $S \rightarrow \bullet \text{NP VP}, [0,0]$, the predictor adds the following states to chart $[0]$:

$$\text{NP} \rightarrow \bullet \text{Det Nominal}, [0,0]$$

$$\text{NP} \rightarrow \bullet \text{Noun}, [0,0]$$

$$\text{NP} \rightarrow \bullet \text{Pronoun}, [0,0]$$

$$\text{NP} \rightarrow \bullet \text{Det Noun PP}, [0,0]$$

Scanner

A scanner is used when a state has a part-of-speech category to the right of the dot. The scanner examines the input to see if the part-of-speech appearing to the right of the dot matches one of the part-of-speech associated with the current input. If yes, then it creates a new state using the rule that allows generation of the input word with this part-of-speech. It advances the pointer over the predicted input category and adds it to the next chart entry. If the state is $A \rightarrow \dots \bullet a \dots, [i, j]$ and 'a' is one of the part-of-speech associated with w_j , then it adds $a \rightarrow \dots w_j \bullet, [i, j]$ to chart $[j+1]$.

Returning to our example, when the state $\text{NP} \rightarrow \bullet \text{Det Nominal}, [0,0]$ is processed, the parser finds a part-of-speech category next to the dot. It checks if the category of the current word (`curr_word`) matches with the expectation in the current state. If yes, then it adds the new state $\text{Det} \rightarrow \text{curr_word} \bullet, [0,1]$ to the next chart entry.

Completer

The completer is used when the dot reaches the right end of the rule. The presence of such a state signifies successful completion of the parse of some grammatical category. The completer identifies all previously generated states that expect this grammatical category at this position in the input and creates new states by advancing the dots over the expected category. All these newly generated states are inserted in the current chart entry. More formally, if $A \rightarrow \dots\bullet, [j, k]$, then the completer adds $B \rightarrow \dots A\bullet \dots [i, k]$ to chart $[k]$ for all states $B \rightarrow \dots\bullet A \dots [i, j]$ in chart $[j]$. An item is added to a set only if it is not already in the set.

Example 4.1 Let us trace the algorithm using sentence (4.7). The sequence of states created by the parser is shown in Figure 4.10.

Chart [0]	S0	$S' \rightarrow \bullet S$	[0,0]
Dummy		start	state
	S1	$S \rightarrow \bullet NP VP$	
	S2	$S \rightarrow \bullet VP$	
	S3	$NP \rightarrow \bullet Det Nominal$	
	S4	$NP \rightarrow \bullet Noun$	
	S5	$NP \rightarrow \bullet Pronoun$	
	S6	$NP \rightarrow Det Noun PP$	
	S7	$VP \rightarrow \bullet Verb NP$	
	S8	$VP \rightarrow \bullet Verb$	
Chart [1]	S9	$Noun \rightarrow paint \bullet$	[0,1]
	S10	$Verb \rightarrow paint \bullet$	[0,1]
	S11	$NP \rightarrow Noun \bullet$	[0,1]
	S12	$VP \rightarrow Verb \bullet NP$	[0,1]
	S13	$VP \rightarrow Verb \bullet$	[0,1]
	S14	$S \rightarrow NP \bullet VP$	[0,1]
	S15	$NP \rightarrow \bullet Det Nominal$	[1,1]
	S16	$NP \rightarrow \bullet Noun$	[1,1]
	S17	$S \rightarrow VP \bullet$	[0,1]
	S18	$VP \rightarrow \bullet Verb NP$	[1,1]
	S19	$VP \rightarrow \bullet Verb$	[1,1]
Chart [2]	S20	$Det \rightarrow the \bullet$	[1,2]
	S21	$NP \rightarrow Det \bullet Nominal$	[1,2]
	S22	$Nominal \rightarrow \bullet Noun$	[2,2]
	S23	$Nominal \rightarrow \bullet Noun Nominal$	[2,2]
Chart [3]	S24	$Noun \rightarrow door \bullet$	[2,3]
	S25	$Nominal \rightarrow Noun \bullet$	[2,3]
	S26	$NP \rightarrow Det Nominal \bullet$	[1,3]
	S27	$S \rightarrow NP \bullet VP$	[0,3]
	S28	$VP \rightarrow Verb NP \bullet$	[0,3]
	S29	$VP \rightarrow \bullet Verb NP$	[3,3]
	S30	$VP \rightarrow \bullet Verb$	[3,3]
	S31	$S \rightarrow VP \bullet$	[0,3]

Figure 4.10 Sequence of states created in parsing sentence (4.7) using Earley algorithm

The presence of the state $S \rightarrow VP \bullet, [0,3]$ in the chart indicates successful completion of the parse of the sentence, but it does not return the exact parse. Thus, the algorithm in this form can be used only to recognize a sentence. However, it is possible to utilize the entries appearing in the table to construct the parse tree of a valid sentence. In order to do so, the algorithm needs to be modified. This modification is carried out by the completer which creates new states by advancing earlier incomplete states whenever it finds a state having a dot at the end of the rule. We add a pointer to the previous states of the new state. A list of previous states is thus maintained. Extracting a parse tree from the chart involves following these pointers, beginning with the state marking the successful completion of the parse. Though the Earley algorithm fills the chart entry in polynomial time, extracting all parse trees still requires an exponential amount of time.

The Earley parser can be augmented with unification structures to eliminate ill-formed structures as they are introduced. This requires certain modification in the algorithm. The first change involves the addition of a feature structure derived from their unification constraints. The second change is the addition of a new field to the chart. This new field is a directed acyclic graph representing the feature structure corresponding to the state. The details of the modified Earley algorithm can be found in Jurafsky and Martin (2000).

4.4.5 The CYK Parser

Like the Earley algorithm, the CYK (Cocke-Younger-Kasami) is a dynamic programming parsing algorithm. However, it follows a bottom-up approach in parsing. It builds a parse tree incrementally. Each entry in the table is based on previous entries. The process is iterated until the entire sentence has been parsed. The CYK parsing algorithm assumes the grammar to be in Chomsky normal form (CNF). A CFG is in CNF if all the rules are of only two forms:

$$A \rightarrow BC$$

$$A \rightarrow w, \text{ where } w \text{ is a word.}$$

The algorithm first builds parse trees of length one by considering all rules which could produce words in the sentence being parsed. Then, it constructs the most probable parse for all the constituents of length two. The parse of shorter constituents constructed in earlier iterations can be used in constructing the parse of longer constituents.

Let us now define PCFG. A PCFG is defined by the pair (G, f) , where G is a CFG and f is a positive function defined over the set of rules such that, the sum of the probabilities associated with the rules expanding a particular non-terminal is 1 (Infante-Lopez and Maarten de Rijke 2000).

$$\sum_{\alpha} f(A \rightarrow \alpha) = 1$$

An example of PCFG is shown in Figure 4.13. We can verify that for each non-terminal, the sum of probabilities is 1.

$$f(S \rightarrow NP VP) + f(S \rightarrow VP) = 1$$

$$f(NP \rightarrow Det Noun) + f(NP \rightarrow Noun) + f(NP \rightarrow Pronoun) + f(NP \rightarrow Det Noun PP) = 1$$

$$f(VP \rightarrow Verb NP) + f(VP \rightarrow Verb) + f(VP \rightarrow VP PP) = 1.0$$

$$f(Det \rightarrow this) + f(Det \rightarrow that) + f(Det \rightarrow a) + f(Det \rightarrow the) = 1.0$$

$$f(Noun \rightarrow paint) + f(Noun \rightarrow door) + f(Noun \rightarrow bird) + f(Noun \rightarrow hole) = 1.0$$

$S \rightarrow NP VP$	0.8	$Noun \rightarrow door$	0.25
$S \rightarrow VP$	0.2	$Noun \rightarrow bird$	0.25
$NP \rightarrow Det Noun$	0.4	$Noun \rightarrow hole$	0.25
$NP \rightarrow Noun$	0.2	$Verb \rightarrow sleeps$	0.2
$NP \rightarrow Pronoun$	0.2	$Verb \rightarrow sings$	0.2
$NP \rightarrow Det Noun PP$	0.2	$Verb \rightarrow open$	0.2
$VP \rightarrow Verb NP$	0.5	$Verb \rightarrow saw$	0.2
$VP \rightarrow Verb$	0.3	$Verb \rightarrow paint$	0.2
$VP \rightarrow VP PP$	0.2	$Preposition \rightarrow from$	0.3
$PP \rightarrow Preposition NP$	1.0	$Preposition \rightarrow with$	0.25
$Det \rightarrow this$	0.2	$Preposition \rightarrow on$	0.2
$Det \rightarrow that$	0.2	$Preposition \rightarrow to$	0.25
$Det \rightarrow a$	0.25	$Pronoun \rightarrow she$	0.35
$Det \rightarrow the$	0.35	$Pronoun \rightarrow he$	0.35
$Noun \rightarrow paint$	0.25	$Pronoun \rightarrow they$	0.25

Figure 4.13 A probabilistic context-free grammar (PCFG)

Similarly, the condition is satisfied for the other non-terminals.

4.5.1 Estimating Rule Probabilities

The next question is how are probabilities assigned to rules? One way to estimate probabilities for a PCFG is to manually construct a corpus of a parse tree for a set of sentences, and then estimate the probabilities of each rule being used by counting them over the corpus. The MLE estimate for a rule $A \rightarrow \alpha$ is given by the expression

$$P_{MLE}(A \rightarrow \alpha) = \frac{\text{Count}(A \rightarrow \alpha)}{\sum_{\alpha} \text{Count}(A \rightarrow \alpha)}$$

If our training corpus consists of two parse trees (as shown in Figure 4.14), we will get the estimates as shown in Table 4.5 for the rules.

Table 4.5 The MLE for the grammar rules used in trees of Figure 4.14

Rule	Count ($A \rightarrow \alpha$)	Count A	MLE estimates
$S \rightarrow VP$	2	2	1
$NP \rightarrow \text{Det Noun PP}$	1	4	0.25
$NP \rightarrow \text{Det Noun}$	3	4	0.75
$VP \rightarrow \text{Verb NP}$	2	3	0.66
$VP \rightarrow VP PP$	1	3	0.33
$\text{Det} \rightarrow \text{the}$	2	2	1
$\text{Noun} \rightarrow \text{hole}$	2	4	0.5
$\text{Noun} \rightarrow \text{door}$	2	4	0.5
$\text{Prep} \rightarrow \text{with}$	1	1	1
$\text{Verb} \rightarrow \text{Paint}$	1	1	1

We now turn to another important question—what do we do with these probabilities? We assign a probability to each parse tree ϕ of a sentence s . The probability of a complete parse is calculated by multiplying the probabilities for each of the rules used in generating the parse tree:

$$P(\phi, s) = \prod_{n \in \phi} p\{r(n)\}$$

where n is a node in the parse tree ϕ and r is the rule used to expand n .

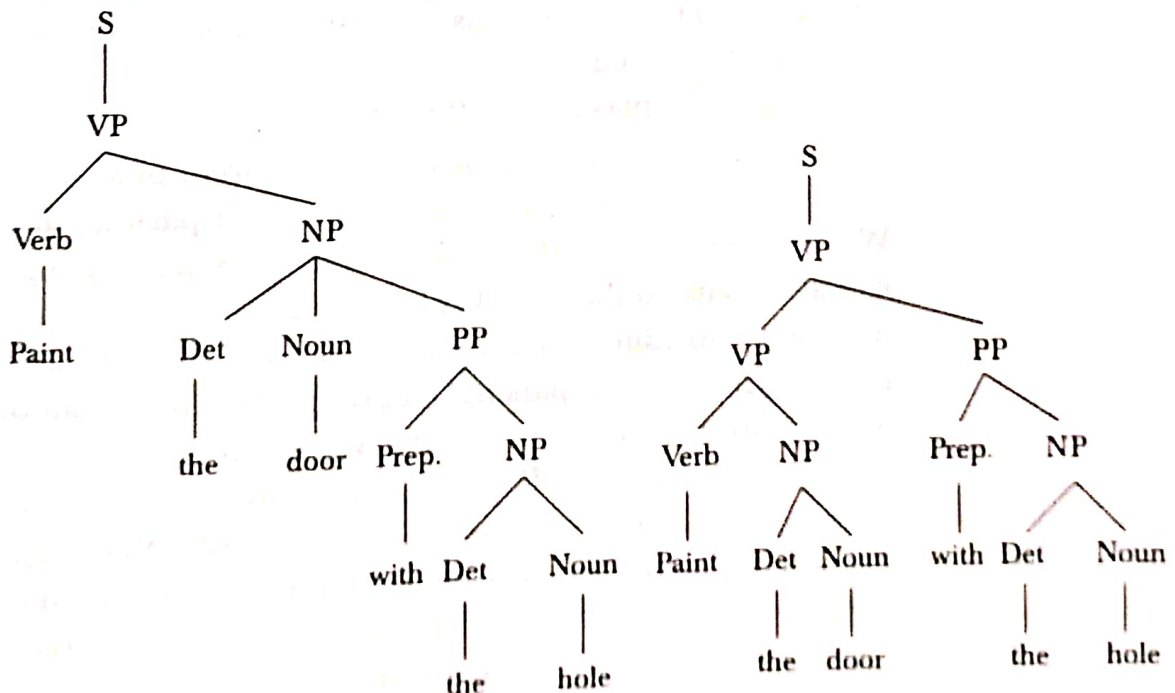


Figure 4.14 Two possible parse trees

For a sentence, the parse trees generated by a PCFG are the same as those generated by a corresponding CFG. However, the PCFG assigns a probability to each parse. The probability of the two parse trees of the sentence *Paint the door with the hole* (shown in Figure 4.14) can be computed as follows:

$$P(t1) = 0.2 * 0.5 * 0.2 * 0.2 * 0.35 * 0.25 * 1.0 * 0.25 * 0.4 * 0.35 * 0.2 \\ = 0.0000030625$$

$$P(t2) = 0.2 * 0.2 * 0.5 * 0.2 * 0.4 * 0.35 * 0.25 * 1 * 0.25 * 0.4 * 0.35 * 0.2 \\ = 0.000001225$$

In this example, the first tree has a higher probability leading to correct interpretation.

We can calculate probability to a sentence s by summing up probabilities of all possible parses associated with it.

$$P(s) = \sum_{t \in \tau(s)} P(t, s)$$

$$= \sum_{t \in \tau(s)} P(t)$$

Thus, the sentence will have the probability

$$P(t1) + P(t2) = 0.0000030625 + 0.000001225 \\ = 0.0000042875$$

4.5.2 Parsing PCFGs

Given a PCFG, a probabilistic parsing algorithm assigns the most likely parse ϕ' to a sentence s .

$$\phi' = \operatorname{argmax}_{T \in \tau(s)} P(T | S)$$

where $\tau(s)$ is the set of all possible parse trees of s .

We have already discussed a number of parsing algorithms for CFG. We will now discuss the probabilistic CYK parsing algorithm. We have already discussed the basic CYK parsing algorithm. Its probabilistic version was first introduced by Ney (1991) and is generally preferred over probabilistic Earley parsing algorithm due to its simplicity. We begin with notations that we use in the algorithm.

As in the basic CYK parsing algorithm, $w = w_1 w_2 w_3 \dots w_i \dots w_j \dots w_n$ represents a sentence consisting of n words. Let $\phi[i, j, A]$ represent the maximum probability parse for a constituent with non-terminal A spanning the words $i, i+1$, up to $i+j-1$. This means it is a sub-tree rooted at A that derives sequence of j words beginning at position i and has a probability greater than all other possible sub-trees deriving the same word sequence.

An array named BP is used to store back pointers. These pointers allow us to recover the best parse. The output for a successful parse is the maximum probability parse $\phi[1,n,S]$, which corresponds to a parse tree rooted at S and spanning the sequence of words $w_1 w_2 w_3 \dots w_n$ i.e. the whole sentence.

```

Initialization:
  for  $i := 1$  to  $n$  do
    for all rules  $A \rightarrow w_i$  do
       $\phi[i,1,A] = P(A \rightarrow w_i)$ 

Recursive Step:
  for  $j = 2$  to  $n$  do
    for  $i = 1$  to  $n-j+1$  do
      begin
         $\phi[i,1,A] = \phi$ 
        for  $k = 1$  to  $j-1$  do
           $\phi'[i,j,A] = \max_k \phi'[i,k,B] \times \phi'[k,j,C] \times P(A \rightarrow BC)$ ,
          such that  $A \rightarrow BC$  is a production rule in grammar
         $BP[i,j,A] = \{ k, A, B \}$ 
      end
    end
  end

```

Figure 4.15 Probabilistic CYK algorithm

The algorithm is given in Figure 4.15. Like the basic CYK algorithm, the first step is to generate items of length 1. However, in this case, we have to initialize the maximum probable parse trees deriving a string of length 1, with the probabilities of the terminal derivation rules used to derive them.

$$\phi[i,1,A] = P(A \rightarrow w_i)$$

Similarly, recursive step involves breaking a string into all possible ways and identifying the maximum probable parse.

$$\phi'[i,j,A] = \max_k \phi'[i,k,B] \times \phi'[k,j,C] \times P(A \rightarrow BC)$$

The rest of the steps follow those of basic CYK parsing algorithm.

4.5.3 Problems with PCFG

The PCFG is not without disadvantages. Its first problem lies in the independence assumption. We calculate the probability of a parse tree assuming that the rules are independent of each other. However, this is not true. How a node expands depends on its location in the parse tree. For example, Francis et al. (1999) showed that pronouns occur more

frequently as subjects rather than objects. These dependencies are captured by a PCFG, as the probability of, say, expanding an NP as pronoun versus a lexical NP, is independent of whether the NP appears as a subject or an object.

Another problem associated with a PCFG is its lack of sensitivity to lexical information. Lexical information plays a major role in determining the correct parse in case of PP attachment ambiguities and coordination ambiguities (Collins 1999). Two structurally different parses that use the same rules will have the same probability under a PCFG, making it difficult to identify the correct or most probable parse. The word appearing in a parse may make certain parses unnatural. This however requires a model which captures lexical dependency statistics for different words. Such a model is presented next.

Lexicalization

In PCFG, the chance of a non-terminal expanding using a particular rule is independent of the actual words involved. However, this independence assumption does not seem reasonable. Words do affect the choice of the rule. Investigations made on tree bank data suggest that the probabilities of various common sub-categorization frames differ depending on the verb that heads the verb phrase (Manning and Schutze 1999). This suggests the need for lexicalization, i.e., involvement of actual words in the sentences, to decide the structure of the parse tree. Lexicalization is also helpful in choosing phrasal attachment positions. This model of lexicalization is based on the idea that there are strong lexical dependencies between heads and their dependents, for example, between a head noun and its modifiers, or between a verb and a noun phrase object, where the noun phrase object in turn can be approximated by its head noun.

One way to achieve lexicalization is to mark each phrasal node in a parse tree by its head word. Figure 4.16 is an example of such a tree. One way to implement this model is to use a lexicalized grammar. A lexicalized grammar is a grammar in which every finite structure is associated with one or more lexical heads. A context free grammar is not lexicalized as no lexical item is associated with its rule, such as, $S \rightarrow NP VP$. Nor is the PCFG lexicalized. In order to convert a PCFG into lexicalized grammar, each of its rules must identify a head daughter, which is one of the constituents appearing on its right hand side, for example head daughter of $S \rightarrow NP VP$ rule is VP. The head word of a node in the parse tree is set to the head word of its head daughter. For example, the head of a verb phrase is the main verb. Hence, the head of the node VP in the Figure 4.16 is jumped. Similarly, the head of a constituent expanded using the rule $S \rightarrow NP VP$ is the head of VP.

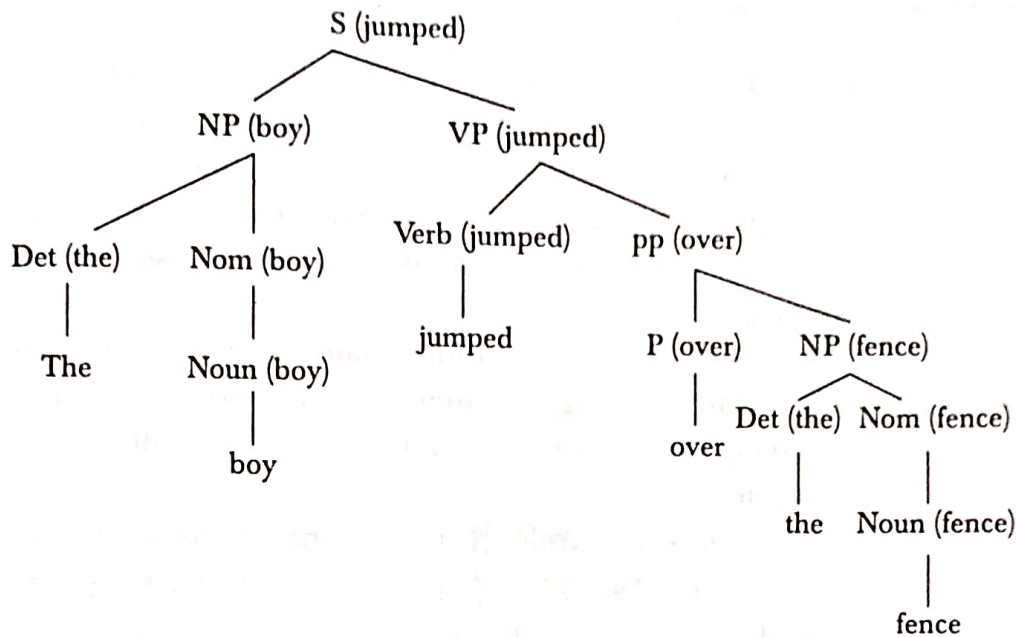


Figure 4.16 A lexicalized tree

The probability in a lexicalized PCFG is calculated for each rule or head of a phrase and head of the sub-phrase or head of the phrase (i.e., head/mother head) combination. For example, we need to collect the following rule/head probability:

VP (jumped) → verb (jumped) PP (over)

VP (jumped) → verb (jumped) PP (into)

VP (jumped) → verb (jumped) PP (to)

An example of the head or mother head combination is the following type of probability:

“What is the probability that an NP whose mother’s head is *over* has the head *fence*?”

With a lexicalized PCFG, the probability of the parse tree is computed as the product of the probability of the head of the constituent c given the probability of the mother $m(c)$ and the probability of the rule used to expand constituent c given the head of constituent (Charniak 1997):

$$P(\phi, s) = \prod_{c \in \phi} p[(h(c)|m(c))] \cdot p[(r(c)|h(c))]$$

where $h(c)$ = head of the constituent c

$r(c)$ = rule used to expand constituent c

and $m(c)$ = mother of the constituent c

4.6 INDIAN LANGUAGES

Not all natural languages have the same characteristics. So far we have talked only of the English language, using CFG as the grammar formalism.